

M1 MIAGE — Frameworks Web Avancés

TP — Application de Gestion de Bibliothèque

avec Angular 17+ & json-server

CHATTI Ichraf — Avril 2026

Contexte du projet

Vous êtes développeur frontend dans une petite bibliothèque municipale. Le bibliothécaire a besoin d'une application web moderne pour gérer son catalogue de livres.

L'application devra permettre de :

- Consulter la liste des livres disponibles
- Ajouter, modifier et supprimer un livre (CRUD complet)
- Rechercher un livre par titre ou par auteur
- Naviguer entre les différentes sections de l'application
- Remplir un formulaire d'ajout avec validation des données

L'API sera simulée avec json-server — aucun backend à écrire de votre côté.

Mise en place de l'environnement

Objectif : Installer json-server, initialiser le projet Angular et vérifier que tout fonctionne avant d'écrire la moindre ligne de code métier.

Étape 0 — Prérequis

Node.js (version 18 ou supérieure) doit être installé. Vérifiez avec : `node -v`

Angular CLI doit être installé. Si ce n'est pas encore fait :

```
npm install -g @angular/cli
```

Étape 1 — Installer et démarrer json-server

json-server permet de simuler une API REST complète à partir d'un simple fichier JSON.

Installez json-server globalement :

```
npm install -g json-server
```

Créez un fichier db.json à la racine de votre futur projet Angular avec le contenu suivant :

```
{
  "livres": [
    { "id": 1, "titre": "Clean Code", "auteur": "Robert C. Martin", "annee": 2008,
      "disponible": true },
    { "id": 2, "titre": "The Pragmatic Programmer", "auteur": "David Thomas", "annee":
      1999, "disponible": true },
    { "id": 3, "titre": "Design Patterns", "auteur": "Gang of Four", "annee": 1994,
      "disponible": false },
    { "id": 4, "titre": "Refactoring", "auteur": "Martin Fowler", "annee": 2018,
      "disponible": true },
    { "id": 5, "titre": "You Don't Know JS", "auteur": "Kyle Simpson", "annee": 2014,
      "disponible": true }
  ]
}
```

Démarrez le serveur depuis ce dossier :

```
json-server --watch db.json --port 3000
```

Vérification

Ouvrez <http://localhost:3000/livres> dans votre navigateur. Vous devez voir le tableau JSON des 5 livres. Si c'est le cas, l'API est prête — laissez ce terminal ouvert.

Étape 2 — Créer le projet Angular

Ouvrez un second terminal et créez le projet Angular :

```
ng new bibliotheque --standalone --routing --style=css
```

Accédez au dossier et lancez le serveur de développement :

```
cd bibliotheque
ng serve
```

Ouvrez <http://localhost:4200> — vous devez voir la page d'accueil Angular par défaut.

Deux terminaux

Vous devez maintenir les deux terminaux ouverts tout au long du TP : l'un pour json-server (port 3000) et l'autre pour Angular (port 4200).

Étape 3 — Structure de l'application à construire

Voici l'arborescence cible que vous allez construire au fil des 5 parties :

```
src/app/
├── components/
└── navbar/
```

```
|   ├── liste-livres/
|   ├── detail-livre/
|   └── formulaire-livre/
├── services/
|   └── livre.service.ts
├── models/
|   └── livre.model.ts
└── app.routes.ts
```

PARTIE 1 — Composants & Data Binding

Durée estimée : 1h30 | Cours de référence : Cours 2



Objectifs de cette partie

À la fin de cette partie, vous saurez :

- Créer et organiser des composants Angular standalone
- Utiliser l'interpolation {{ }}, le property binding [] et l'event binding ()
- Afficher une liste de données avec *ngFor
- Définir un modèle TypeScript typé (interface Livre)

Étape 1 — Créer le modèle Livre

Créez le fichier `src/app/models/livre.model.ts` et définissez l'interface TypeScript :

```
// src/app/models/livre.model.ts
export interface Livre {
  id?: number;          // optionnel : non défini lors de la création
  titre: string;
  auteur: string;
  annee: number;
  disponible: boolean;
}
```

💡 Le champ `id` est marqué optionnel (`id?`) car il sera généré automatiquement par `json-server` lors de l'ajout d'un nouveau livre.

Étape 2 — Générer les composants

Générez les 4 composants principaux avec Angular CLI depuis le dossier `bibliotheque` :

```
ng g c components/navbar
ng g c components/liste-livres
ng g c components/detail-livre
ng g c components/formulaire-livre
```

Dans VS Code, vérifiez que le dossier `src/app/components` contient bien 4 sous-dossiers, chacun avec ses fichiers `.ts`, `.html` et `.css`.

Étape 3 — Composant NavBar

Dans `navbar.component.ts`, déclarez la propriété `titre` :

```
export class NavbarComponent {
  titre: string = 'BiblioApp';
}
```

Dans `navbar.component.html`, créez la barre de navigation :

```
<nav>
  <h1>{{ titre }}</h1>
  <span>Bibliothèque Municipale</span>
</nav>
```

Étape 4 — Composant ListeLivres (données statiques)

Dans `liste-livres.component.ts`, déclarez une liste de livres codée en dur (elle sera remplacée par l'API en Partie 3) :

```
import { Livre } from '../models/livre.model';

livres: Livre[] = [
  { id: 1, titre: 'Clean Code', auteur: 'Robert C. Martin', annee: 2008, disponible: true },
  { id: 2, titre: 'The Pragmatic Programmer', auteur: 'David Thomas', annee: 1999, disponible: true },
];
```

```
{ id: 3, titre: 'Design Patterns', auteur: 'Gang of Four', annee: 1994, disponible: false },
];
```

Dans `liste-livres.component.html`, affichez la liste avec `*ngFor` et colorez le statut de disponibilité :

```
<div class="liste">
  <div *ngFor="let livre of livres" class="carte">
    <h3>{{ livre.titre }}</h3>
    <p>{{ livre.auteur }} - {{ livre.annee }}</p>
    <span [class]="livre.disponible ? 'dispo' : 'indispo'">
      {{ livre.disponible ? 'Disponible' : 'Indisponible' }}
    </span>
  </div>
</div>
```

⚠ Import de CommonModule requis

Pour que `*ngFor` fonctionne dans un composant standalone, vous devez importer `CommonModule` dans le tableau `imports` du décorateur `@Component` de `liste-livres.component.ts`.

Étape 5 — Intégrer les composants dans AppComponent

Modifiez `app.component.ts` pour afficher la navbar et le router-outlet :

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { NavbarComponent } from '../components/navbar/navbar.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, NavbarComponent],
  template: `
    <app-navbar></app-navbar>
    <main class="container">
      <router-outlet></router-outlet>
    </main>
  `,
  styles: [`.container { max-width: 1100px; margin: 80px auto 0; padding: 20px; }`]
})
export class AppComponent {}
```

💬 Questions de réflexion

1. Quelle est la différence entre `{{ livre.titre }}` et `[class]="..."` ?
L'un est de l'interpolation (affichage de texte), l'autre du property binding (liaison d'une propriété HTML à une expression TypeScript).
2. Que se passerait-il si vous oubliez d'importer `CommonModule` dans un composant standalone utilisant `*ngFor` ?



Résultat attendu : une page avec une barre de navigation affichant "BiblioApp", et une liste de 3 livres codée en dur avec le statut de disponibilité coloré (vert / rouge).

PARTIE 2 — Routing & Navigation

Durée estimée : 1h30 | Cours de référence : Cours 3

Objectifs de cette partie

À la fin de cette partie, vous saurez :

- Configurer le système de routing Angular avec des routes paramétrées
- Utiliser routerLink et routerLinkActive dans les templates
- Récupérer un paramètre d'URL avec ActivatedRoute
- Mettre en place le Lazy Loading pour une route

Étape 1 — Configurer les routes

Dans app.routes.ts, définissez les routes de l'application :

```
// src/app/app.routes.ts
import { Routes } from '@angular/router';
import { ListeLivresComponent } from '../components/liste-livres/liste-livres.component';
import { DetailLivreComponent } from '../components/detail-livre/detail-livre.component';

export const routes: Routes = [
  { path: '', redirectTo: 'livres', pathMatch: 'full' },
  { path: 'livres', component: ListeLivresComponent },
  { path: 'livres/:id', component: DetailLivreComponent },
  {
    path: 'ajouter',
    loadComponent: () =>
      import('../components/formulaire-livre/formulaire-livre.component')
        .then(m => m.FormulairelivreComponent) // Lazy Loading
  },
  {
    path: 'modifier/:id',
    loadComponent: () =>
      import('../components/formulaire-livre/formulaire-livre.component')
        .then(m => m.FormulairelivreComponent) // Lazy Loading
  },
  { path: '**', redirectTo: 'livres' }
];
```

💡 Le Lazy Loading (loadComponent) permet de ne charger le composant FormulairelivreComponent que quand l'utilisateur navigue vers /ajouter ou /modifier/:id, améliorant les performances au démarrage.

Étape 2 — Mettre à jour la NavBar

Dans navbar.component.html, ajoutez les liens de navigation avec routerLink et routerLinkActive :

```
<nav>
  <h1>{{ titre }}</h1>
  <ul>
    <li><a routerLink="/livres" routerLinkActive="actif">Catalogue</a></li>
    <li><a routerLink="/ajouter" routerLinkActive="actif">+ Ajouter un livre</a></li>
  </ul>
</nav>
```

Dans navbar.component.ts, ajoutez RouterLink et RouterLinkActive aux imports :

```
imports: [RouterLink, RouterLinkActive]
```

Étape 3 — Rendre les cartes cliquables

Dans liste-livres.component.html, ajoutez un bouton "Voir le détail" sur chaque carte :

```
<button [routerLink]="['/livres', livre.id]">Voir le détail</button>
<button [routerLink]="['/modifier', livre.id]">Modifier</button>
```

N'oubliez pas d'importer RouterLink dans le tableau imports de liste-livres.component.ts.

Étape 4 — Composant DetailLivres

Dans detail-livre.component.ts, récupérez l'id depuis l'URL avec ActivatedRoute :

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, RouterLink } from '@angular/router';
import { CommonModule } from '@angular/common';

@Component({
  standalone: true,
  imports: [CommonModule, RouterLink],
  ...
})
export class DetailLivresComponent implements OnInit {
  livreId: number = 0;

  constructor(private route: ActivatedRoute) {}

  ngOnInit(): void {
    this.livreId = Number(this.route.snapshot.paramMap.get('id'));
    console.log('ID du livre :', this.livreId);
  }
}
```

Dans detail-livre.component.html, affichez temporairement l'id récupéré :

```
<h2>Détail du livre #{{ livreId }}</h2>
<p>Les données complètes seront chargées depuis l'API en Partie 3.</p>
<button routerLink="/livres">← Retour à la liste</button>
```

Questions de réflexion

1. Quelle est la différence entre route.snapshot.paramMap.get() et route.paramMap.subscribe() ? Lequel faudrait-il utiliser si la route pouvait changer sans rechargement du composant ?
2. Que fait le { path: '**', redirectTo: 'livres' } dans app.routes.ts ?



Résultat attendu : navigation fonctionnelle entre les 3 pages (liste, détail, formulaire). L'URL change à chaque navigation sans rechargement. L'id s'affiche dans le titre de la page détail.

PARTIE 3 — Services & Appels HTTP

Durée estimée : 2h | Cours de référence : Cours 5



Objectifs de cette partie

À la fin de cette partie, vous saurez :

- Créer un service Angular injectable pour centraliser les appels HTTP
- Consommer une API REST avec HttpClient (GET, POST, PUT, DELETE)
- Utiliser les Observables et subscribe() pour traiter les réponses asynchrones
- Gérer les états de chargement (isLoading) et les erreurs

Étape 1 — Activer HttpClient

Dans app.config.ts, ajoutez provideHttpClient() aux providers :

```
// src/app/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient() // ← ajouter cette ligne
  ]
};
```

Étape 2 — Créer le LivreService

Générez le service avec Angular CLI :

```
ng g s services/livre
```

Implémentez les 5 méthodes CRUD dans livre.service.ts :

```
// src/app/services/livre.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Livre } from '../models/livre.model';

@Injectable({ providedIn: 'root' })
export class LivreService {
  private apiUrl = 'http://localhost:3000/livres';

  constructor(private http: HttpClient) {}

  getLivres(): Observable<Livre[]> {
    return this.http.get<Livre[]>(this.apiUrl);
  }

  getLivreById(id: number): Observable<Livre> {
    return this.http.get<Livre>(`${this.apiUrl}/${id}`);
  }

  ajouterLivre(livre: Livre): Observable<Livre> {
    return this.http.post<Livre>(this.apiUrl, livre);
  }

  modifierLivre(id: number, livre: Livre): Observable<Livre> {
```



```

    return this.http.put<Livre>(`${this.apiUrl}/${id}`, livre);
  }

  supprimerLivre(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}

```

Étape 3 — Connecter ListeLivres au service

Remplacez les données en dur dans `liste-livres.component.ts` par des données réelles :

```

livres: Livre[] = [];
isLoading: boolean = true;
erreur: string = '';

constructor(private livreService: LivreService) {}

ngOnInit(): void {
  this.livreService.getLivres().subscribe({
    next: (data) => {
      this.livres = data;
      this.isLoading = false;
    },
    error: (err) => {
      this.erreur = 'Impossible de charger les livres. Vérifiez que json-server est démarré.';
      this.isLoading = false;
    }
  });
}

```

Mettez à jour le template pour afficher le spinner et les messages d'erreur :

```

<div *ngIf="isLoading" class="spinner">Chargement en cours...</div>
<div *ngIf="erreur" class="erreur">{{ erreur }}</div>
<div *ngIf="!isLoading && !erreur" class="liste">
  <!-- votre *ngFor ici -->
</div>

```

Étape 4 — Connecter DetailLivre au service

Dans `detail-livre.component.ts`, chargez le livre complet par son id :

```

livre: Livre | null = null;

constructor(private route: ActivatedRoute, private livreService: LivreService) {}

ngOnInit(): void {
  const id = Number(this.route.snapshot.paramMap.get('id'));
  this.livreService.getLivreById(id).subscribe({
    next: (data) => this.livre = data,
    error: () => this.livre = null
  });
}

```

Dans le template, affichez les informations complètes du livre :

```

<div *ngIf="livre; else notFound">
  <h2>{{ livre.titre }}</h2>
  <p><strong>Auteur :</strong> {{ livre.auteur }}</p>
  <p><strong>Année :</strong> {{ livre.annee }}</p>
  <p><strong>Statut :</strong> {{ livre.disponible ? 'Disponible' : 'Indisponible' }}</p>
  <button [routerLink]="['/modifier', livre.id]">Modifier</button>
  <button routerLink="/livres">← Retour à la liste</button>
</div>

```

```
</div>
<ng-template #notFound><p>Livre introuvable.</p></ng-template>
```

Étape 5 — Suppression depuis la liste

Ajoutez un bouton Supprimer sur chaque carte dans le template de ListeLivres :

```
<button (click)="supprimerLivres(livre.id!)">Supprimer</button>
```

Implémentez la méthode dans le composant :

```
supprimerLivres(id: number): void {
  if (confirm('Supprimer ce livre définitivement ?')) {
    this.livreService.supprimerLivres(id).subscribe(() => {
      this.livres = this.livres.filter(l => l.id !== id);
    });
  }
}
```

Questions de réflexion

1. Pourquoi utilise-t-on subscribe() plutôt que async/await avec les Observables Angular ?
2. Que se passe-t-il si json-server n'est pas démarré quand vous ouvrez l'application ?
Comment votre gestion d'erreur doit-elle réagir ?
3. Pourquoi filtre-t-on le tableau local (this.livres.filter) après la suppression plutôt que de rappeler getLivres() ?



Résultat attendu : l'application lit les données depuis json-server. La suppression fonctionne et met à jour la liste sans rechargement. Le détail d'un livre s'affiche avec toutes ses informations.

PARTIE 4 — Formulaires Réactifs & Validation

Durée estimée : 2h | Cours de référence : Cours 4



Objectifs de cette partie

À la fin de cette partie, vous saurez :

- Créer un formulaire réactif avec FormBuilder et FormGroup
- Ajouter des validateurs (required, minLength, min, max)
- Afficher des messages d'erreur contextuels selon les règles de validation
- Gérer les deux modes ajout et modification dans un même composant

Étape 1 — Importer ReactiveFormsModule

Dans formulaire-livre.component.ts, ajoutez les imports nécessaires :

```
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from
 '@angular/forms';
import { Router, ActivatedRoute, RouterLink } from '@angular/router';
import { LivreService } from '../services/livre.service';

@Component({
  standalone: true,
  imports: [ReactiveFormsModule, RouterLink, CommonModule],
  ...
})
```

Étape 2 — Construire le FormGroup

Dans la classe du composant, déclarez le formulaire et détectez le mode (ajout ou modification) :

```
livreForm!: FormGroup;
isModification: boolean = false;
livreId: number | null = null;

constructor(
  private fb: FormBuilder,
  private livreService: LivreService,
  private router: Router,
  private route: ActivatedRoute
) {}

ngOnInit(): void {
  // Initialisation du formulaire avec ses validateurs
  this.livreForm = this.fb.group({
    titre: ['', [Validators.required, Validators.minLength(2)]],
    auteur: ['', [Validators.required, Validators.minLength(3)]],
    annee: ['', [Validators.required, Validators.min(1800),
Validators.max(2030)]],
    disponible: [true]
  });

  // Si un id est présent dans l'URL → mode modification
  const id = this.route.snapshot.paramMap.get('id');
  if (id) {
    this.isModification = true;
    this.livreId = Number(id);
    this.livreService.getLivreById(this.livreId).subscribe(livre => {
      this.livreForm.patchValue(livre); // pré-remplissage du formulaire
    });
  }
}
```

Étape 3 — Template du formulaire

Dans `formulaire-livre.component.html`, créez le formulaire avec les messages d'erreur contextuels :

```
<h2>{{ isModification ? 'Modifier le livre' : 'Ajouter un nouveau livre' }}</h2>

<form [formGroup]="livreForm" (ngSubmit)="onSubmit()">

  <div class="champ">
    <label for="titre">Titre *</label>
    <input id="titre" type="text" formControlName="titre" />
    <span class="erreur-champ"
      *ngIf="livreForm.get('titre')?.invalid && livreForm.get('titre')?.touched">
      Le titre est requis (minimum 2 caractères).
    </span>
  </div>

  <div class="champ">
    <label for="auteur">Auteur *</label>
    <input id="auteur" type="text" formControlName="auteur" />
    <span class="erreur-champ"
      *ngIf="livreForm.get('auteur')?.invalid && livreForm.get('auteur')?.touched">
      Le nom de l'auteur est requis (minimum 3 caractères).
    </span>
  </div>

  <div class="champ">
    <label for="annee">Année de publication *</label>
    <input id="annee" type="number" formControlName="annee" />
    <span class="erreur-champ"
      *ngIf="livreForm.get('annee')?.invalid && livreForm.get('annee')?.touched">
      L'année doit être comprise entre 1800 et 2030.
    </span>
  </div>

  <div class="champ">
    <label>
      <input type="checkbox" formControlName="disponible" />
      Disponible immédiatement
    </label>
  </div>

  <div class="actions">
    <button type="submit" [disabled]="livreForm.invalid">
      {{ isModification ? 'Enregistrer les modifications' : 'Ajouter le livre' }}
    </button>
    <button type="button" routerLink="/livres">Annuler</button>
  </div>

</form>
```

Étape 4 — Méthode onSubmit()

Dans la classe du composant, implémentez la soumission selon le mode :

```
onSubmit(): void {
  if (this.livreForm.invalid) return;

  const livre: Livre = this.livreForm.value;

  if (this.isModification && this.livreId) {
```

```
this.livreService.modifierLivre(this.livreId, livre).subscribe(() => {  
  this.router.navigate(['/livres']);  
});  
} else {  
  this.livreService.ajouterLivre(livre).subscribe(() => {  
    this.router.navigate(['/livres']);  
  });  
}  
}
```

Questions de réflexion

1. Pourquoi le bouton Submit est-il désactivé avec `[disabled]="livreForm.invalid"` ?
Que se passe-t-il si vous supprimez cette contrainte ?
2. Quelle est la différence entre `patchValue()` et `setValue()` ?
Dans quel cas l'un est-il préférable à l'autre ?
3. Quel est l'avantage d'utiliser un seul composant `FormulairelivrComponent` pour les deux modes ajout et modification ?



Résultat attendu : le formulaire valide les champs, affiche les erreurs au toucher. L'ajout crée un livre dans json-server. La modification pré-remplit le formulaire et met à jour le livre existant.

PARTIE 5 — Fonctionnalités Avancées & Finalisation

Durée estimée : 2h | Cours de référence : Cours 2 (approfondissement)



Objectifs de cette partie

À la fin de cette partie, vous saurez :

- Implémenter une barre de recherche en temps réel avec two-way binding
- Utiliser les pipes Angular intégrés (uppercase, titlecase)
- Appliquer des styles dynamiques avec ngClass et ngStyle
- Soigner l'interface CSS pour rendre l'application professionnelle

Étape 1 — Barre de recherche en temps réel

Dans `liste-livres.component.ts`, ajoutez la propriété recherche et un getter filtré :

```
import { FormsModule } from '@angular/forms'; // pour [(ngModel)]

recherche: string = '';

get livresFiltres(): Livre[] {
  const terme = this.recherche.toLowerCase();
  return this.livres.filter(l =>
    l.titre.toLowerCase().includes(terme) ||
    l.auteur.toLowerCase().includes(terme)
  );
}
```

Dans le template, ajoutez le champ de recherche et utilisez `livresFiltres` dans `*ngFor` :

```
<div class="recherche">
  <input type="text" [(ngModel)]="recherche"
    placeholder="Rechercher par titre ou auteur..." />
  <p>{{ livresFiltres.length }} résultat(s) trouvé(s)</p>
</div>

<div *ngFor="let livre of livresFiltres" class="carte">
  <!-- ... -->
</div>
```

Étape 2 — Utiliser les pipes Angular

Améliorez l'affichage des données avec les pipes intégrés :

```
<!-- Titre en majuscules -->
<h3>{{ livre.titre | uppercase }}</h3>

<!-- Auteur avec initiales en majuscules -->
<p>{{ livre.auteur | titlecase }}</p>
```

Étape 3 — Styles dynamiques avec ngClass et ngStyle

Remplacez `[class]` simple par `ngClass` pour gérer plusieurs classes CSS :

```
<div [ngClass]="{
  'carte': true,
  'disponible': livre.disponible,
  'indisponible': !livre.disponible
}">
```

Utilisez `ngStyle` pour colorer le statut dynamiquement :

```
<span [ngStyle]="{
  color: livre.disponible ? 'green' : 'red',
  fontWeight: 'bold'
}">
```

```
{{ livre.disponible ? 'Disponible' : 'Indisponible' }}  
</span>
```

Étape 4 — Soigner le CSS

Cette étape est obligatoire. Une interface soignée fait partie des critères d'évaluation. Voici les règles minimales à respecter :

- Une grille CSS (grid ou flexbox) pour la liste de cartes
- Couleurs distinctes pour les livres disponibles (vert) et indisponibles (rouge)
- Style visible pour les messages d'erreur du formulaire (rouge, italique)
- Champ de recherche stylé (bordure, padding, largeur adaptée)
- Boutons avec hover effect

Questions de réflexion

1. Quelle est la différence entre `[class]="..."` et `[ngClass]="..."` ?
Dans quel cas `ngClass` est-il plus adapté ?
2. Comment pourriez-vous créer un pipe personnalisé pour afficher
"publié il y a N ans" à partir de l'année de publication ?
(Bonus — voir section Bonus ci-dessous)



Résultat attendu : la recherche filtre en temps réel, les pipes formatent les données, les styles s'appliquent dynamiquement. L'interface est propre et fonctionnelle de bout en bout.

Dépannage — Erreurs fréquentes

Voici les erreurs les plus courantes rencontrées lors de ce TP et comment les résoudre.

Message d'erreur	Solution
<code>Can't bind to 'ngFor'</code>	Ajouter CommonModule dans les imports du composant standalone.
<code>Can't bind to 'formGroup'</code>	Ajouter ReactiveFormsModule dans les imports du composant.
<code>NullInjectorError: HttpClient</code>	Ajouter provideHttpClient() dans app.config.ts.
<code>CORS error / ERR_CONNECTION_REFUSED</code>	Vérifier que json-server est lancé sur le port 3000.
<code>NG0302: component not found</code>	Vérifier que le composant est bien dans le tableau imports du composant parent.
<code>ERROR TypeError: Cannot read properties of null</code>	Utiliser *ngIf="livre" avant d'accéder aux propriétés d'un objet potentiellement null.

Bonus (non obligatoire)

Ces fonctionnalités supplémentaires sont pour les étudiants qui ont terminé en avance. Elles ne sont pas notées mais valorisées.

- Pagination : afficher 5 livres par page avec des boutons Précédent / Suivant
- Filtre de disponibilité : boutons radio "Tous / Disponibles / Indisponibles"
- Message de confirmation visuel : bannière verte après ajout ou modification réussis
- Pipe personnalisé : afficher "publié il y a N ans" à partir de l'année de publication
- Route Guard : demander confirmation avant de quitter un formulaire modifié non sauvegardé
- Tri de la liste : par titre (alphabétique) ou par année (croissant/décroissant)

Bilan & Critères d'évaluation

L'application complète doit couvrir l'ensemble des fonctionnalités suivantes. La grille est transmise en début de TP pour que vous puissiez vous auto-évaluer au fil de votre progression.

Fonctionnalité	Cours associé	Points
Affichage de la liste des livres (ngFor, interpolation)	Cours 2 — Composants	3
Navigation entre les pages (routerLink, routes paramétrées)	Cours 3 — Routing	3
Chargement des données depuis l'API (HttpClient, subscribe)	Cours 5 — Services	4
CRUD complet : ajout, modification, suppression	Cours 5 — HttpClient	4
Formulaire réactif avec validation	Cours 4 — Reactive Forms	4
Barre de recherche en temps réel	Cours 2 — Data Binding	2
Pipes (uppercase, titlecase) et directives (ngClass, ngStyle)	Cours 2 — Directives	2
Gestion des erreurs & états de chargement	Cours 5 — Services	2

Fonctionnalité	Cours associé	Points
Qualité du code (typage TypeScript, organisation)	Cours 1 — Angular	2
TOTAL		26

Rendu attendu

Déposez sur Moodle (ou envoyez par mail) :

- Le code source complet du projet Angular (dossier src/)
- Le fichier db.json avec au moins 5 livres
- Un fichier README.md expliquant comment lancer le projet
- Un dépôt GitHub (public ou privé avec accès partagé) ou une archive ZIP

Commandes à documenter dans le README :

```
Terminal 1 : json-server --watch db.json --port 3000
```

```
Terminal 2 : ng serve
```